



À propos			
Date de création	Nom du projet	Commanditaires	Supervisé par:
28 juin 2026	Web cloner	NGANGA Georgia & MOUANGA Gloire	M.Nickel MOUNZIEHO

1. Présentation générale

a) Contexte et Description

Web Cloner est un outil hybride combinant une interface en ligne de commande (CLI) et une interface utilisateur web (Web UI). Il permet de cloner fidèlement et localement n'importe quel site web (des pages statiques simples aux applications dynamiques complexes de type SPA) en capturant l'intégralité de ses assets (CSS, JS, images, polices) et en interceptant ses flux API de manière transparente.

b) Objectifs Principaux

- **Fidélité visuelle et comportementale** : Rendu local identique à l'original (gestion des interactions et du responsive).
- **Capture Réseau Exhaustive** : Interception complète (XHR, Fetch, GraphQL, WebSockets).
- **Mode Hybride Intelligent** : Rejeu des appels API dynamiques en local via un *Service Worker*.
- **Automatisation Avancée** : Détection automatique des frameworks modernes, gestion de l'hydratation et stratégies anti-bot.

c) Profil des Utilisateurs Cibles

Web Cloner s'adresse principalement aux **développeurs web**, aux **data scientists** et aux **analystes en Business Intelligence (BI)** ayant besoin d'accéder à des répliques locales et autonomes d'applications complexes. Les développeurs et ingénieurs logiciels l'utilisent pour auditer l'architecture de sites existants, analyser des structures de frameworks modernes ou concevoir des interfaces en mode hors-ligne. Parallèlement, les professionnels de la data et de l'automatisation l'exploitent comme un outil avancé de collecte et d'ingénierie de données, leur permettant de capturer des flux applicatifs réels, de cartographier des réponses API et de travailler sur des environnements simulés sans dépendre d'une connexion réseau active ou de contraintes de serveurs distants.

2. Spécifications Fonctionnelles

a) Moteur de navigation et stratégies de capture

Le cœur de l'application s'appuie sur une navigation *headless* pilotée via **Playwright**.

- **Anti-Détection** : Intégration native d'un mode *stealth* via playwright-stealth pour contourner les protections de base.
- **Stratégies de chargement** : Configurations optionnelles des critères d'attente (`domcontentloaded`, `networkidle`, `load`, ou sélecteur CSS spécifique).
- **Interactions Simulées** : Déclenchement automatique du scroll progressif infini (avec détection de stabilisation), ouverture des accordéons/dropdowns et clics automatisés sur les boutons de type ("*Load more*" / "*Voir plus*").
- **Gestion de Sessions** : Prise en charge des cookies et gestion des formulaires d'authentification (login).

b) Support multi-Frameworks (SPA) & Hydratation

L'outil embarque un moteur de détection unifié (`detect_spa_framework`) capable d'identifier le framework utilisé et d'attendre son hydratation complète pour figer le DOM au meilleur moment :

- **Angular** : Détection via `detect_angular_version()` (analyse de l'attribut `ng-version`). Attente de la stabilisation de la zone via `wait_for_angular_hydration()`.
- **Nuxt & Next.js** : Attente dédiée via `wait_for_nuxt_hydration()`.
- **Vue, React, Ember, Svelte** : Détection des objets globaux fenêtres ou des attributs racines spécifiques (`data-v-app`, `data-reactroot`).

c) Injection et gestion spécifique de Tailwind CSS

Pour garantir la fidélité visuelle et éviter les ruptures de styles dues au *purge* ou au chargement dynamique de Tailwind sur certains sites:

- **Extraction** : Analyse et capture des feuilles de style externes contenant le mot-clé `tailwind` et des balises `<style type="tailwind">`.

- **Fix Mobile Viewport** : Réécriture forcée des balises meta viewport (width=device-width, initial-scale=1.0) pour bloquer la compression d'URL sur les frameworks réactifs.
- **Nettoyage CDN** : Neutralisation des scripts tiers intrusifs (ex: fslightbox, scripts de préchargement de polices Tailwind) via l'ajout d'attributs factices (data-dummy-fix).

d) Modes de clonage

Mode	Description	Comportement Technique
Auto (détection) — recommandé	Détection et adaptation automatique au site cible.	Analyse le site à la volée (detect_spa_framework) et applique la meilleure stratégie d'attente, d'hydratation et de capture sans intervention utilisateur.
Nuxt Perfect (animations+polices)	Profil de capture optimisé pour le framework Nuxt/Vue.	Active les routines spécifiques wait_for_nuxt_hydration, capture les transitions d'animations CSS/JS et force la mise en cache complète des polices locales.
Snapshot (DOM figé)	Capture brute de l'état du document à un instant précis.	Extrait le contenu de page.content() immédiatement après la stabilisation de la page, figeant le DOM exact généré par le JavaScript de manière immuable.
Screenshot (capture canvas)	Reconstruction graphique basée sur l'aspect visuel.	Capture les rendus d'écrans ou les éléments complexes non textuels (comme les graphiques <canvas> dynamiques, WebGL, ou dashboards) pour les restituer fidèlement.
Hybride (rejeu API) — défaut	Clonage complet avec simulation dynamique du backend.	HTML + Assets + Génération d'un Service Worker local chargé d'intercepter et de rejouer de manière transparente les requêtes API enregistrées (api_map.json).
Statique (HTML + assets)	Extraction traditionnelle et légère.	Téléchargement strict du code HTML de base et des ressources de style et de structure associés (CSS, JS, images). Aucun rejeu de requêtes réseau ou d'API.

3. Architecture technique & base de code

a) Flux de clonage

- **Navigation** : Playwright ouvre la page avec la stratégie d'attente configurée
- **Interception** : NetworkInterceptor capture toutes les requêtes (ressources + API)
- **Attente** : wait_strategy + wait_for_content() + wait_for_selector()
- **Interactions** : Scroll infini, cookies, accordéons, dropdowns, hover, Load more
- **Capture** : page.content() → HTML final après scroll et interactions
- **Sauvegarde** : StorageManager trie et sauvegarde dans _clones/<domaine>_<timestamp>/
- **Réécriture** : HTMLRewriter réécrit toutes les URLs en chemins locaux
- **Téléchargement** : ResourceDownloader télécharge les ressources distantes
- **Routes SPA** : collect_spa_routes() → ajoute les routes découvertes à la file
- **Finalisation** : Génération de manifest.json, url_map.json, api_map.json, service-worker.js

b) Structure du Projet (web_cloner/)

```
web_cloner/
├── main.py          # Point d'entrée unique (CLI & Routage UI)
├── requirements.txt  # Dépendances du projet (Python 3.13)
├── .env             # Variables d'environnement (Clé API Groq)
├── core/            # Modules Noyaux (Core)
│   ├── browser.py   # Gestion et configuration de Playwright & Stealth
│   ├── crawler.py    # Orchestration du crawling, file d'attente et configuration
│   ├── downloader.py # Téléchargement asynchrone des assets avec Rate Limiter
│   ├── interactions.py # PageInteractor (Scroll, Hydratation SPA, Correctifs Tailwind)
│   ├── interceptor.py # NetworkInterceptor (Capture XHR/Fetch/GraphQL)
│   ├── rewriter.py   # HTMLRewriter & CSSRewriter (Localisation des URLs)
│   ├── storage.py    # StorageManager & Gérant du Manifeste
│   └── utils.py      # Helpers génériques (Logs, formateurs de chaînes)
├── utils/           # Modules Utilitaires
│   ├── cli.py        # Interface CLI (Rich console, spinners, bannières)
│   └── groq.py        # GroqSummarizer (Modèle llama-3.3-70b-versatile)
├── webui/           # Interface Graphique Web
│   ├── server.py     # Serveur web backend (FastAPI + WebSocket)
│   ├── templates/
│   │   └── index.html # Interface UI Glassmorphism Sombre
│   └── static/
│       ├── script.js  # Logique client WebSocket & Mises à jour UI
│       └── style.css   # Design system Dark Glassmorphism
```

c) Structure d'un dossier de sortie (_clones/)

Chaque site web cloné est isolé sous la forme : (_clones/<domaine>_<timestamp>/)

```
├── pages/           # Fichiers HTML structurels (index.html, etc.)
├── styles/          # Feuilles de styles CSS localisées
├── scripts/         # Fichiers JavaScript
├── images/          # Médias graphiques (PNG, JPG, SVG, WebP)
├── fonts/           # Polices de caractères (WOFF, WOFF2, TTF)
├── media/           # Fichiers vidéo/audio lourds (MP4, MP3)
├── api/             # Payloads et réponses des appels API capturés
├── url_map.json      # Table de correspondance : URL distante ↔ Chemin local
├── api_map.json      # Table de correspondance : Requête API ↔ Fichier JSON local
├── manifest.json     # Métadonnées du clone (Framework détecté, date, statistiques)
├── service-worker.js # Script d'interception pour le rejeu des APIs en mode Hybride
├── sw-register.js    # Script d'enregistrement automatique du Service Worker
└── resume.md         # Analyse et résumé textuel du site cloné
```

4. Interfaces & Utilisation

a) Interface en ligne de commande (CLI)

L'interface en ligne de commande s'appuie sur la librairie **Rich** pour fournir des retours visuels clairs (tableaux de statistiques, barres de progression).

Syntaxe de base

```
python main.py https://example.com
```

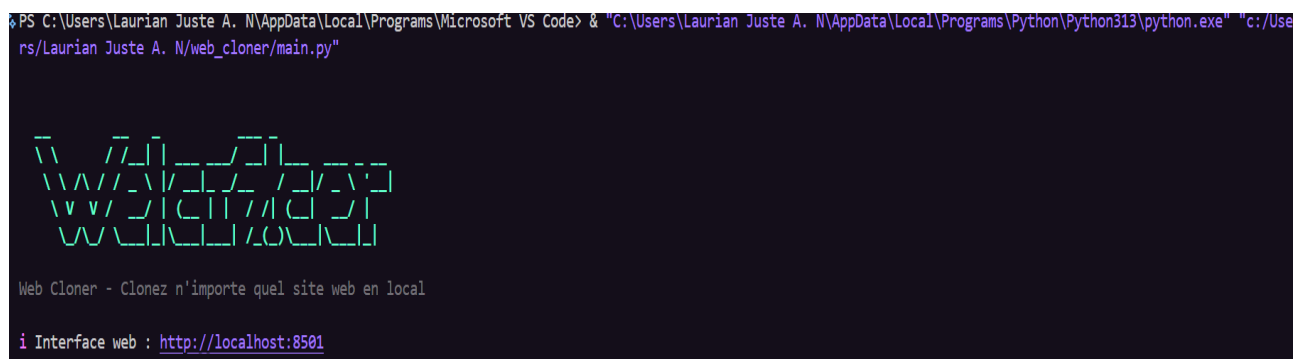
Clonage en mode statique avec ciblage précis et limitation de profondeur

```
python main.py https://example.com --clone-mode static --max-pages 3 --no-interactions
```

Activation du mode furtif et d'une stratégie d'attente réseau

```
python main.py https://example.com --stealth --wait-strategy networkidle --page-timeout 120000
```

Génération du résumé du site cloné



```
PS C:\Users\Laurian Juste A. N\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Laurian Juste A. N\AppData\Local\Programs\Python\Python313\python.exe" "c:/Users/Laurian Juste A. N/web_cloner/main.py"

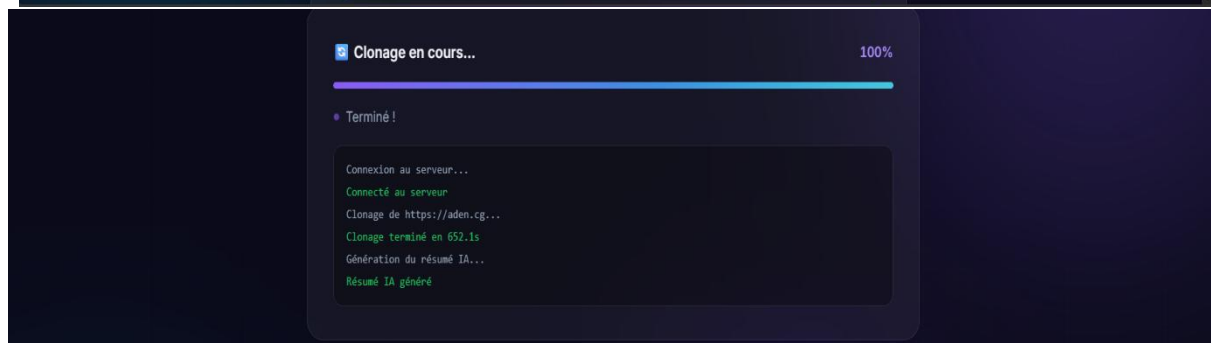
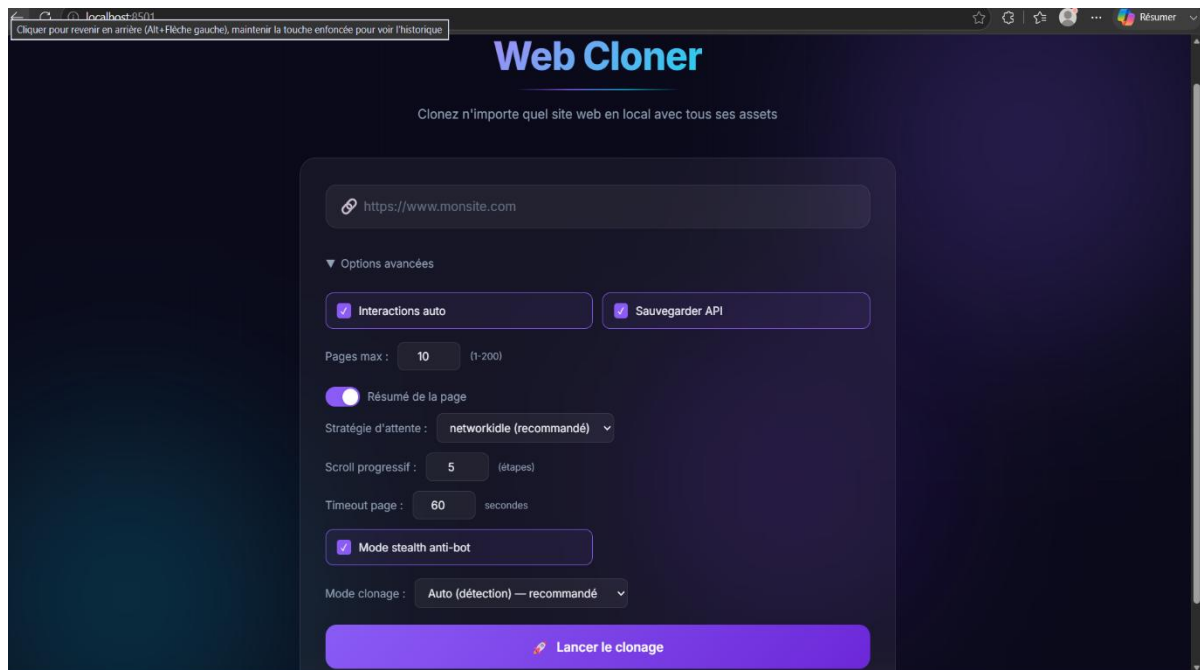
Web Cloner - Clonez n'importe quel site web en local

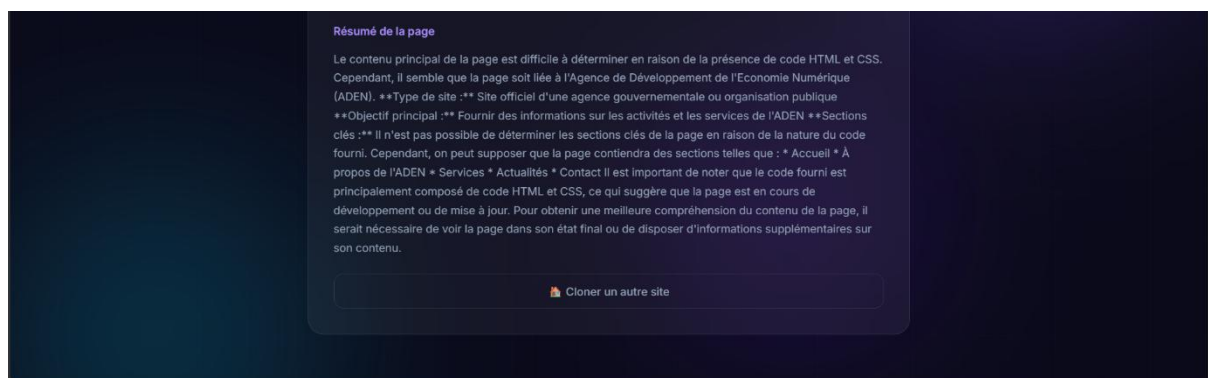
i Interface web : http://localhost:8501
```

b) Interface Web (Web UI)

- **Design :** Mode sombre (*Dark theme*) basé sur une esthétique *glassmorphism* avec fonds animés de dégradés et de particules.
- **Fonctionnalités :**
 - Formulaire d'URL avec validation en temps réel.
 - Panneau de configuration dépliant pour les options avancées (sélection des modes, activation du mode *stealth*, timeouts).

- Suivi en direct de la progression via une connexion **WebSocket** (flux de logs catégorisés : info, success, warning, error, download).
- Affichage des métriques finales (taille totale du clone, framework détecté, nombre de requêtes interceptées).
- Composant d'affichage pour le **Résumé de la page**.





5. Matrice des Options de Configuration

Option	Type	Défaut	Description
--ui	flag	-	Lancer l'interface web
--port	int	8501	Port pour l'interface web
--no-open	flag	-	Ne pas ouvrir le navigateur
-o / --output	str	-	Dossier de sortie personnalisé
--max-pages	int	10	Nombre max de pages à cloner
--no-interactions	flag	-	Désactiver les interactions auto
--no-api	flag	-	Ne pas sauvegarder les appels API
--headless	flag	True	Mode headless
--no-headless	flag	-	Afficher le navigateur
--verbose / -v	flag	-	Mode verbeux
--groq-key	str	-	Clé API Groq
--summarize	flag	-	Générer un résumé IA
--overwrite	flag	-	Écraser le dossier existant
--user-agent	str	-	User-Agent personnalisé
--stealth	flag	-	Mode stealth anti-bot
--wait-strategy	str	networkidle	Stratégie d'attente
--wait-selector	str	-	Sélecteur CSS à attendre
--page-timeout	int	60000	Timeout navigation (ms)
--scroll-steps	int	5	Étapes de scroll
--clone-mode	str	hybrid	Mode clonage (static/hybrid)

6. Technologies utilisées

Technologie	Version	Rôle
Python	3.13	Langage principal
Playwright	1.60	Navigation headless, capture réseau
playwright-stealth	2.0	Anti-détection headless
FastAPI	0.110	Serveur web + WebSocket
Uvicorn	0.29	Serveur ASGI
Jinja2	3.1	Templating HTML
BeautifulSoup4	4.12	Parsing HTML
aiohttp	3.9	Téléchargement asynchrone
Rich	13.0	CLI (bannière, spinner, tableaux)
Groq	0.5	API IA pour résumé
python-dotenv	1.0	Variables d'environnement

7. Limitations Actuelles & Risques

Le processus de clonage dynamique repose sur l'exécution de navigateurs automatisés et la simulation d'infrastructures réseaux. Cette approche impose des limites techniques et architecturales indépassables, ainsi que des contraintes strictes sur les ressources de la machine hôte.

a) Limitations applicatives & protocolaires

- **Verrous Sécuritaires Anti-Bot :** Les pare-feux applicatifs (WAF) stricts (Cloudflare, Akamai) analysent le comportement (vitesse, micro-mouvements) et la réputation des adresses IP. L'automatisation peut être détectée et bloquée malgré le mode *stealth*.
- **Protocoles Sans État & Flux Dynamiques :** Les flux **WebSockets** et SSE (canaux bidirectionnels et asynchrones en temps réel) ne peuvent pas être cartographiés de manière statique.

- Les modules d'authentification **OAuth 2.0 / OIDC** (Social Login) échouent localement en raison du changement de domaine (localhost), bloqués par les règles de sécurité CORS.
- **Médias Fragmentés (Streaming HLS/DASH)** : Les vidéos découpées en centaines de micro-segments éphémères (.ts, .m4s) avec jetons d'accès temporaires ne peuvent pas être consolidées en un fichier local unique.
- **Mutations de Données (POST, PUT, DELETE)** : Le jeu local via Service Worker simule les requêtes de lecture (GET) mais ne possède aucune base de données dynamique pour traiter les modifications d'état du côté serveur.

b) Impacts Matériels & Contraintes de Ressources (RAM / CPU)

L'utilisation intensive de Playwright en mode Headless ou Visible génère une charge critique sur l'architecture matérielle locale :

- **Saturation Progressive de la RAM** : Chaque page analysée instancie un processus de navigation complet (Chromium/Edge). Sur des sites dotés d'un **Scroll Infini Extrême** (> 50 étapes), la mémoire RAM s'accumule rapidement pour maintenir le DOM en cache. De plus, les frameworks SPA récents (Nuxt, React,..) utilisent le *Virtual Scrolling* (destruction des éléments du haut de page pour économiser la mémoire du site d'origine), ce qui peut provoquer des pertes de données structurelles dans la capture finale si la RAM locale s'essouffle à stabiliser l'arborescence.
- **Consommation CPU et Fuites de Mémoire** : L'interception systématique de l'intégralité du trafic réseau (scripts, polices, requêtes XHR), couplée à l'injection à la volée de scripts de contournement (Tailwind fixes, DOM snapshots), sollicite fortement les processeurs multi-cœurs. En cas de crawling multi-pages simultané (profondeur

élevée), le manque de ressources matérielles peut entraîner des ralentissements majeurs du serveur local FastAPI ou des crashes inopinés (*Out of Memory*) du script d'orchestration.

c) **Rendus Graphiques Basés sur Canvas (HTML5 Canvas / WebGL)**

Les applications reposant principalement sur les éléments <canvas> (HTML5 Canvas, WebGL, WebGPU ou moteurs de rendu propriétaires) ne peuvent pas être clonées fidèlement. Contrairement au DOM HTML, un canvas ne contient pas une structure de balises réécrivable mais uniquement un tampon graphique généré en mémoire par JavaScript ou par le GPU. Le clonage peut éventuellement capturer une image statique du rendu final (capture d'écran ou snapshot), mais il est impossible de reconstruire automatiquement la logique de dessin, les animations, les interactions utilisateur ou les scènes 2D/3D qui ont produit cette image. En conséquence, les tableaux de bord interactifs, éditeurs graphiques, jeux navigateur et visualisations scientifiques utilisant intensivement Canvas ou WebGL ne peuvent être reproduits que sous forme de rendu figé.